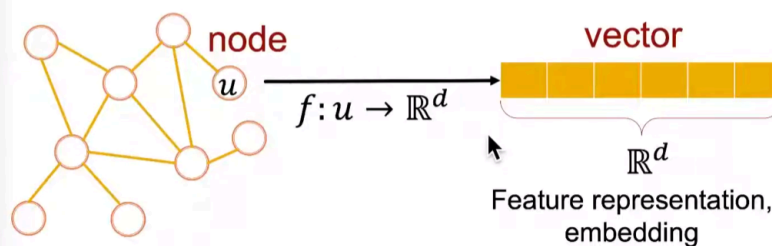


3. Node Embeddings

2026-03-15, 21:50

Tags: [GNN Neural Network](#)

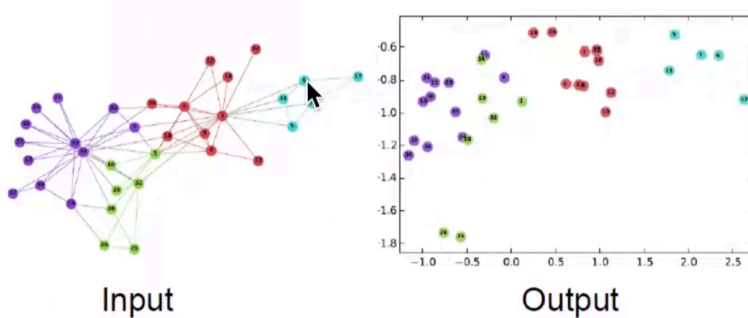
Goal: Efficient task-independent feature learning for machine learning with graphs



Embedding is the feature representation of the graph

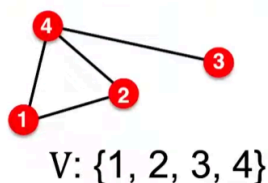
an example will be:

- **2D embedding of nodes of the Zachary's Karate Club network:**



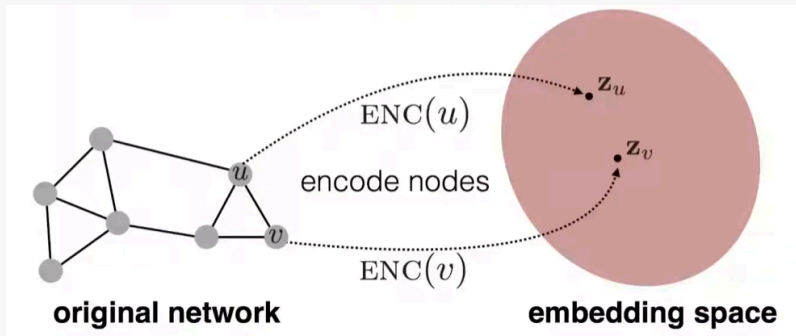
Set Up

- Assume we have a graph G , where
 1. V is the vertex set
 2. A is the adjacency matrix (assume binary)
 3. For simplicity: no node features or extra information is used



$$A = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

We need to ensure the similarity in the embedding space approximate the similarity in graph



- Encoder: maps from nodes to embeddings
- Define a noode similarity function
- Decoder DEC maps from embeddings to the similarity score
- Optimize the parameters of the encoders so that

$$\text{similarity}(u, v) \approx Z_v^T \cdot Z_u$$

- **Encoder:** maps each node to a low-dimensional vector

$$\text{ENC}(v) = \mathbf{z}_v$$

node in the input graph

d -dimensional embedding

- **Similarity function:** specifies how the relationships in vector space map to the relationships in the original network

$$\text{similarity}(u, v) \approx \mathbf{z}_v^T \mathbf{z}_u$$

Similarity of u and v in the original network

Decoder
dot product between node embeddings

Shallow Encoding

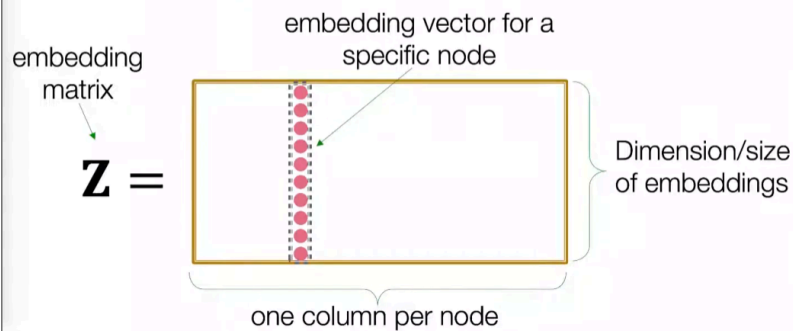
Simplest encoding approach: **Encoder is just an embedding-lookup**

$$\text{ENC}(v) = \mathbf{z}_v = \mathbf{Z} \cdot v$$

$\mathbf{Z} \in \mathbb{R}^{d \times |\mathcal{V}|}$ matrix, each column is a node embedding [what we learn / optimize]

$v \in \mathbb{I}^{|\mathcal{V}|}$ indicator vector, all zeroes except a one in column indicating node v

Simplest encoding approach: **encoder is just an embedding-lookup**



Example

Say you have a small graph with **4 nodes** (A, B, C, D), and you want **3-dimensional embeddings** ($d=3$).

The embedding matrix Z (3×4):

$$Z = \begin{bmatrix} 0.2 & 0.9 & 0.1 & 0.5 \\ 0.8 & 0.3 & 0.7 & 0.2 \\ 0.5 & 0.6 & 0.4 & 0.9 \end{bmatrix}$$

To get the embedding for node B, you create its one-hot vector

$$\begin{aligned} v_B &= [0, 1, 0, 0]^T \\ z_B &= Z \cdot v_B = [0.9, 0.3, 0.6]^T \end{aligned}$$

Define Node Similarity

- Key choice of methods is **how they define node similarity**.
- Should two nodes have a similar embedding if they...
 - are linked?
 - share neighbors?
 - have similar "structural roles"?
- We will now learn node similarity definition that uses **random walks**, and how to optimize embeddings for such a similarity measure.

This is unsupervised/self-supervised way of learning node embeddings

- This is **unsupervised/self-supervised** way of learning node embeddings
 - We are **not** utilizing node labels
 - We are **not** utilizing node features
 - The goal is to directly estimate a set of coordinates (i.e., the embedding) of a node so that some aspect of the network structure (captured by DEC) is preserved
- These embeddings are **task independent**
 - They are not trained for a specific task but can be used for any task.

Random Walk Approach for Node Embedding

Notation Used

- **Vector $\mathbf{z}_u$** :
 - The embedding of node u (what we aim to find).
- **Probability $P(v | \mathbf{z}_u)$** :  Our model prediction based on $\mathbf{z}_u$
 - The **(predicted) probability** of visiting node v on random walks starting from node u .

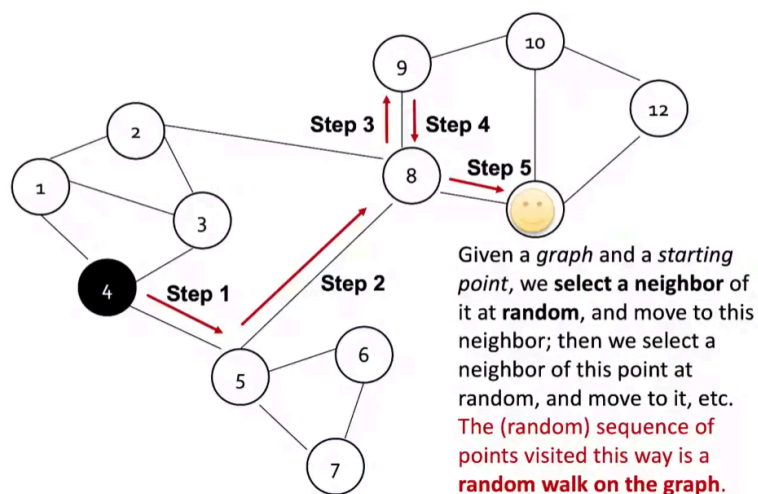
Non-linear functions used to produce predicted **probabilities**

- **Softmax** function
 - Turns vector of K real values (model predictions) into K probabilities that sum to 1: $\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$.
- **Sigmoid** function:
 - S-shaped function that turns real values into the range of $(0, 1)$.
Written as $S(x) = \frac{1}{1+e^{-x}}$.

Random Walk

Given a graph and a starting point, we select a neighbor of it at random, and move to this neighbor; then we select a neighbor of this point at random, and move to it

Basic Logic

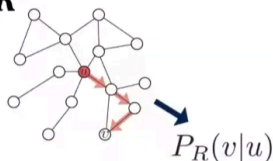


Random-walk embedding

$$\mathbf{z}_u^T \mathbf{z}_v \approx \text{probability that } u \text{ and } v \text{ co-occur on a random walk over the graph}$$

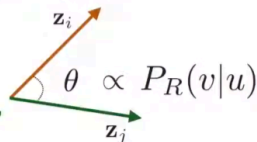
Therefore:

1. **Estimate probability of visiting node v on a random walk starting from node u using some random walk strategy R**



2. **Optimize embeddings to encode these random walk statistics:**

Similarity in embedding space (Here: dot product = $\cos(\theta)$) encodes random walk "similarity"



The reason of using random walk:

1. It is expressive: flexible stochastic definition of node similarity that incorporates both local and higher-order neighborhood information. (Idea: if random walk starting from node u and visit v with high probability, u and v are similar)
2. It is efficiency: Do not need to consider all node pairs when training; only to consider pairs that co-occur on random walks

Unsupervised Feature Learning

- Intuition: Find embedding of nodes in d – *dimensional* space that preserves similarity *(In other word, the idea is to translate the discrete structure of a graph into a cotinuous mathematical space where geometry reflects "closeness" in the network)*
- Idea: Learn node embedding such that nearby nodes are close together in the network

Given a node u , how do we define nearby nodes?

- $N_R(u)$ neighbourhood of u obtained by some random walk strategy R .

- Given $G = (V, E)$,
- Our goal is to learn a mapping $f: u \rightarrow \mathbb{R}^d$:

$$f(u) = \mathbf{z}_u$$

- Log-likelihood objective:

$$\max_f \sum_{u \in V} \log P(N_R(u) | \mathbf{z}_u)$$

- $N_R(u)$ is the neighborhood of node u by strategy R

- Given node u , we want to learn feature representations that are predictive of the nodes in its random walk neighborhood $N_R(u)$

Optimization Process

1. Run **short fixed-length random walks** starting from each node u in the graph using some random walk strategy R
2. For each node u collect $N_R(u)$, the multiset* of nodes visited on random walks starting from u
3. Optimize embeddings according to: **Given node u , predict its neighbors $N_R(u)$**

$$\max_f \sum_{u \in V} \log P(N_R(u) | \mathbf{z}_u) \Rightarrow \text{Maximum likelihood objective}$$

- The math

Equivalently,

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log(P(v|\mathbf{z}_u))$$

- **Intuition:** Optimize embeddings $\mathbf{z}_u$ to maximize the likelihood of random walk co-occurrences
- **Parameterize $P(v|\mathbf{z}_u)$ using softmax:**

$$P(v|\mathbf{z}_u) = \frac{\exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^T \mathbf{z}_n)}$$

Why softmax?

We want node v to be most similar to node u (out of all nodes n).

Intuition: $\sum_i \exp(x_i) \approx \max_i \exp(x_i)$

The softmax function treat the similarity between nodes as a dot product in vector space. An example of such case...

Imagine a tiny network with 3 nodes: **A**, **B**, and **C**. Suppose our random walk strategy says that if we start at **A**, we often visit **B**, but rarely visit **C**.

We want to learn 2D embeddings for them: $\mathbf{z}_A, \mathbf{z}_B, \mathbf{z}_C$.

The Goal:

We want $P(B|A)$ to be high and $P(C|A)$ to be low.

Step 1: Initial (Random) Embeddings

Let's say our model starts with random vectors:

- $\mathbf{z}_A = [1, 0]$
- $\mathbf{z}_B = [0, 1]$ (Perpendicular to A)
- $\mathbf{z}_C = [1, 0.1]$ (Very similar to A)

Step 2: Calculate Dot Products (Scores)

- $\mathbf{z}_A \cdot \mathbf{z}_B = (1)(0) + (0)(1) = 0$
- $\mathbf{z}_A \cdot \mathbf{z}_C = (1)(1) + (0)(0.1) = 1$

Step 3: Compute Softmax Probability for Node B

$$P(B|A) = \frac{e^0}{e^0 + e^1} = \frac{1}{1 + 2.71} \approx 0.27$$

Step 4: The Optimization (The "Learning")

The model sees that the probability for neighbor **B** is only **27%**, which is low. The optimization (using Gradient Descent) will:

1. **Nudge** $\mathbf{z}_B$ to be more like $\mathbf{z}_A$ (increasing the dot product).
2. **Push** further away from $\mathbf{z}_C$ (decreasing the dot product).

Negative Sampling to Simplify the Calculation

- Sample k negative nodes each with prob. proportional to its degree
- Two consideration for k :

1. Higher K gives more robust estimates
2. Higher K corresponds to higher bias on negative events

■ **Solution: Negative sampling**

$$\log\left(\frac{\exp(\mathbf{z}_u^T \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^T \mathbf{z}_n)}\right)$$

$$\approx \log\left(\sigma(\mathbf{z}_u^T \mathbf{z}_v)\right) - \sum_{i=1}^k \log\left(\sigma(\mathbf{z}_u^T \mathbf{z}_{n_i})\right), n_i \sim P_V$$

sigmoid function
(makes each term a "probability" between 0 and 1)

random distribution
over nodes

Instead of normalizing w.r.t. all nodes, just
normalize against k random "negative samples" n_i

Why is the approximation valid?

Technically, this is a different objective. But Negative Sampling is a form of Noise Contrastive Estimation (NCE) which approx. maximizes the log probability of softmax.

New formulation corresponds to using a logistic regression (sigmoid func.) to distinguish the target node v from nodes n_i sampled from background distribution P_v .

More at <https://arxiv.org/pdf/1402.3722.pdf>

Usually people choose between 5~20. It is a super small number if we are thinking about the size of the node numbers

Stochastic Gradient Descent

Instead of evaluating gradients over all examples, evaluate it for each individual training examples

- Initialize $\mathbf{z}_i$ at some randomized value for all i .
- Iterate until convergence: $\mathcal{L}^{(u)} = \sum_{v \in N_R(u)} -\log(P(v|\mathbf{z}_u))$
 - Sample a node i , for all j calculate the derivative $\frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{z}_j}$.
 - For all j , update: $\mathbf{z}_j \leftarrow \mathbf{z}_j - \eta \frac{\partial \mathcal{L}^{(i)}}{\partial \mathbf{z}_j}$.

Node2vec

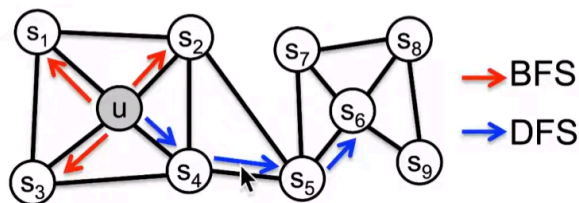
Goal: Embed nodes with similar network neighborhoods close in the feature space

We frame this goal as a maximum likelihood optimization problem, independent to the downstream prediction tasks

Therefore, the key observation: Flexible notion of network neighborhood $N_R(u)$ of node u leads to rich node embeddings. Develop biased 2^{nd} order random walk R to generate network neighborhood $N_R(u)$ of node u .

The idea:

Idea: use flexible, biased random walks that can trade off between **local** and **global** views of the network ([Grover and Leskovec, 2016](#)).



Define different type of node movement:

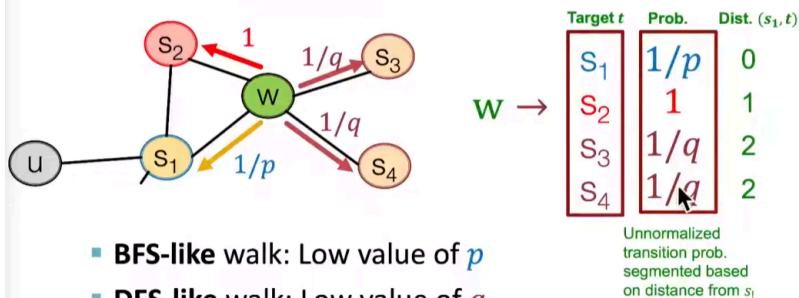
Biased fixed-length random walk R that given a node u generates neighborhood $N_R(u)$

- Two parameters:
 - **Return parameter p :**
 - Return back to the previous node
 - **In-out parameter q :**
 - Moving outwards (DFS) vs. inwards (BFS)
 - Intuitively, q is the “ratio” of BFS vs. DFS

Demonstrate in probability:

- **Walker came over edge (s_1, w) and is at w .**

Where to go next?



- **BFS-like walk:** Low value of p
- **DFS-like walk:** Low value of q

$N_R(u)$ are the nodes visited by the biased walk

Steps of node2vec algorithm:

1. compute random walk probabilities
2. Simulate r random walks of length l starting from each node u
3. Optimize the node2vec objective using gradient descent

Beauty here: it has linear-time complexity

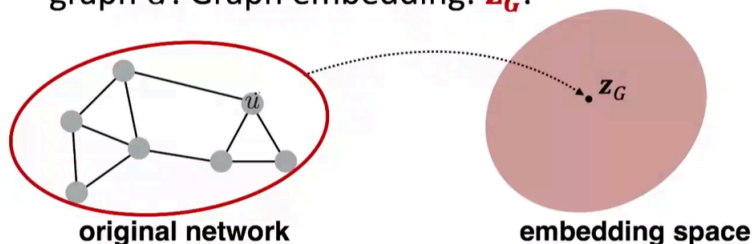
ALL 3 steps are individually parallelizable

Embedding entire graph

Goal: want to embed a subgraph or an entire graph G

Graph embedding: z_G

- **Goal:** Want to embed a subgraph or an entire graph G . Graph embedding: z_G .



- **Tasks:**

- Classifying toxic vs. non-toxic molecules
- Identifying anomalous graphs

Idea 1: Average

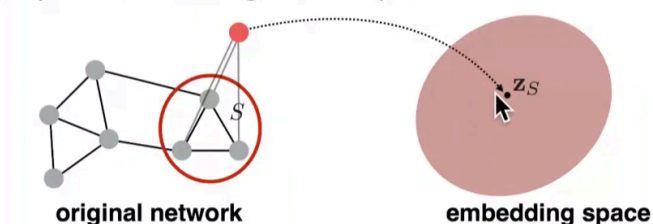
- Run a standard graph embedding technique on the (sub)graph G
- Then just sum (or average) the node embeddings in the (sub)graph G

$$Z_G = \sum_{v \in G} Z_v$$

Basically the average of the value of all the node in the graph. This is used to classify molecules based on their graph structure

Idea 2: Create virtual node

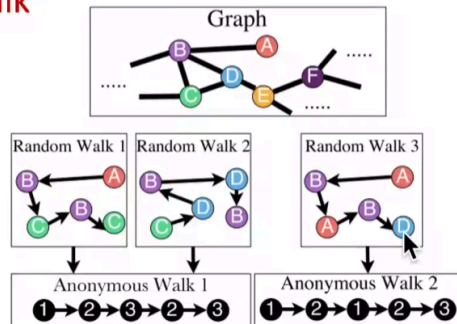
- Introduce a "virtual node" to represent the (sub)graph and run a standard graph embedding technique
 - **Idea 2:** Introduce a **"virtual node"** to represent the (sub)graph and run a standard graph embedding technique



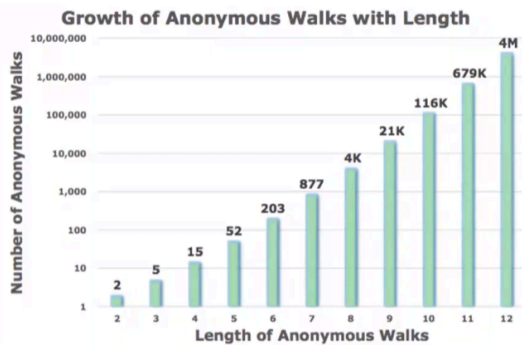
Idea 3: anonymous walk

- States in anonymous walks correspond to the index of the first time we visited the node in a random walk.
- **Standard Walk:** Node A $\rightarrow$ Node B $\rightarrow$ Node A $\rightarrow$ Node C
- **Anonymous Version:** 1 $\rightarrow$ 2 $\rightarrow$ 1 $\rightarrow$ 3 (A is the 1st node seen, B is the 2nd, C is the 3rd).

States in **anonymous walks** correspond to the index of the **first time** we visited the node in a random walk



- Agnostic the identity of the nodes visited (hence anonymous)
- Number of anonymous walks grows exponentially



Number of anonymous walks grows exponentially:

- There are 5 anon. walks w_i of length 3:
 $w_1=111, w_2=112, w_3=121, w_4=122, w_5=123$

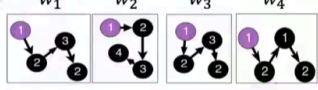
Determine how many anonymous walks are needed

- We want the distribution to have error of more than ϵ with prob. less than :
- The formula determines the minimum number of walks needed to ensure your estimated distribution is close to the "true" distribution within an error bound ϵ :

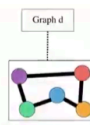
$$m = \frac{2}{\epsilon^2} (\log(2 - 2) - \log())$$

- **(Eta):** The total number of **possible** anonymous walk patterns of a certain length. This grows very quickly as walk length increases.
- **ϵ (Epsilon):** Your tolerance for error. Because it is squared in the denominator (ϵ^2), if you want to be twice as accurate, you need **four times** as many samples.
- **(Delta):** Your failure probability (e.g., $= 0.01$ means you want to be right 99% of the time).
- $\log(2 - 2)$: This term represents the complexity of the "pattern space." Since there are possible patterns, there are 2 possible subsets of patterns to consider.

Idea 4: Learn Walk Embedding

- A vector parameter $\mathbf{z}_G$ for input graph
 - The embedding of entire graph to be learned
- Starting from **node 1**: Sample anonymous random walks, e.g.
 
- Learn to predict walks that co-occur in Δ -size window** (e.g. predict w_2 given w_1, w_3 if $\Delta = 1$)
- Objective:

$$\max \sum_{t=\Delta}^{T-\Delta} \log P(w_t | w_{t-\Delta}, \dots, w_{t+\Delta}, \mathbf{z}_G)$$
- Sum the objective over all nodes in the graph



The process:

Run T different random walks from u each of length l

- Run T different random walks from u each of length l :**

$$N_R(u) = \{w_1^u, w_2^u \dots w_T^u\}$$

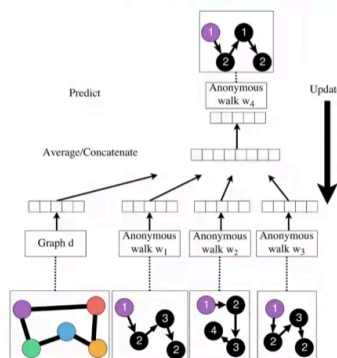
- Learn to predict walks that co-occur in Δ -size window**
- Estimate embedding z_i of anonymous walk w_i
Let η be number of all possible walk embeddings

Objective:
$$\max_{\mathbf{z}, \mathbf{d}} \frac{1}{T} \sum_{t=\Delta}^{T-\Delta} \log P(w_t | \{w_{t-\Delta}, \dots, w_{t+\Delta}, \mathbf{z}_G\})$$

- $P(w_t | \{w_{t-\Delta}, \dots, w_{t+\Delta}, \mathbf{z}_G\}) = \frac{\exp(y(w_t))}{\sum_{i=1}^{\eta} \exp(y(w_i))}$ ← All possible walks (require negative sampling)
- $y(w_t) = b + U \cdot \left(\text{cat}\left(\frac{1}{2\Delta} \sum_{i=-\Delta}^{\Delta} z_i, \mathbf{z}_G\right) \right)$
 - $\text{cat}\left(\frac{1}{2\Delta} \sum_{i=-\Delta}^{\Delta} z_i, \mathbf{z}_G\right)$ means an average of anonymous walk embeddings in window, concatenated with the graph embedding $\mathbf{z}_G$
 - $b \in \mathbb{R}, U \in \mathbb{R}^D$ are learnable parameters. This represents a linear layer.

- We obtain the graph embedding $\mathbf{z}_G$ (learnable parameter) after optimization
- Use $\mathbf{z}_G$ to make predictions (e.g. graph classification)
 - Option1:** Inner product Kernel $\mathbf{z}_{G_1}^T \mathbf{z}_{G_2}$ (Lecture 2)
 - Option2:** Use a neural network that takes $\mathbf{z}_G$ as input to classify

Overall Architecture



How to use Embeddings

■ How to use embeddings z_i of nodes:

- **Clustering/community detection:** Cluster points z_i
- **Node classification:** Predict label of node i based on z_i
- **Link prediction:** Predict edge (i, j) based on (z_i, z_j)
 - Where we can: concatenate, avg, product, or take a difference between the embeddings:
 - Concatenate: $f(z_i, z_j) = g([z_i, z_j])$
 - Hadamard: $f(z_i, z_j) = g(z_i * z_j)$ (per coordinate product)
 - Sum/Avg: $f(z_i, z_j) = g(z_i + z_j)$
 - Distance: $f(z_i, z_j) = g(\|z_i - z_j\|_2)$
- **Graph classification:** graph embedding z_G via aggregating node embeddings or anonymous random walks.
Predict label based on graph embedding z_G